Projekttagebuch von Art FANS

Mitglieder:

Alex Marggraf

Simon Zeugin

Frederic Weyssow

Noah Nellen

1. Treffen: 26.02.2025 12:00-13:00, ZG-Spiegelgasse 1

- Gruppen Namen (Art FANS)

- Spielnamen: War of Arts

- Spielflow: Begriff wird ausgewählt, Zeichenperiode mit variabler Zeitlänge, Zeichnungen bewerten, Leaderboard anzeigen, drei Runden

- Klassen Verteilung: Server Klasse, Client Klasse, Game Klasse, Window Klasse, kleine Player Klasse

- Begriffe werden aus einer lokalen Datenbank geholt und Client Klasse hat Methode «getTheme»

- Fenster mit create / join Lobby, Zeitperiode einstellbar (und Anzahl Runden)

- Fenster mit Chat, Zeichenoberfläche, Timer und Begriff

2. Treffen: 02.03.2025 16:30-17:15, Discord

- Aufgabenverteilung für Projektplan verteilt

3. Treffen 05.03.2025 12:00-13:00, Spiegelgasse 5

- neuer Gruppen Name: Battle Draw

- Präsentation zusammen Durchgegangen und angepasst

4. Treffen 07.03.2025 14:00-15:30, Pharmazentrum

Software Design Client/Server: TCP

Client Klasse:

Attribute: bool sendPing

long lastPing

long lastPong

initaite Methode ()

connect Methode ()

sendMessage Methode ()

recieve/checkMessage Methode ()

createMessage Methode ()

while (true) {

//TODO:

//handlePing (zum Beispiel für einen Disconnect -> if sendPing True and currentTime - lastPing > 10 s)

//handleMessages

}

Server Klasse:

Attribute: ArrayList<int> clients

ArrayList<int> lobbies

initiate Methode ()

createMessage Methode ()

//sendMessage Methode ()

//recieveMessage Methode ()

// TODO: Datenbank integrieren

while (true) {

// TODO:

// listen

// start Threat

}

Thread Klasse:

Attribute: Game gamestate

bool sendPing

long lastPing

long lastPong

sendMessage ()

recieveMessage ()

disconnect Methode ()

while (true) {

// TODO:

// handlePing

// handleMessages

}

Main Klasse / Methode:

```
Client client
client.initiate()
client.connect()
while (true) {
// TODO:
// checkInput
// handleMessages
}
```

Netzwerkprotokoll:

```
Befehl          | param1       | param2
-----------------------------------------------------------------
                |              |
-----------------------------------------------------------------
                |              |
-----------------------------------------------------------------
```

Aufteilung: Frederic und Alex machen den Backbone dieser Klassen

Noah und Simon ergänzen Methoden und implementieren Datenbank

5. Treffen 15.03.2025 13:00-13:30, Discord

Der geschriebene Code von Alex und Fredric wurde erklärt. Sie haben den Code ganz geschrieben -> war schon genug schwierig zu zweit zu kommunizieren. Simon und Noah sind für das Game an sich verantwortlich.

Noah: Software Quality Concept, Netzwerkprotokoll und Project Diary

Simon: Username uniqueness mit Database

Das Software-Design der Server/Client-Architektur musste abgeändert werden aufgrund von unvorhergesehenen Ereignissen. Hauptsächlich werden alle Prozesse, die einen Stream nach Nachrichten abhören, als Thread ausgeführt. Zudem wird das Hören nach neuen Verbindungen auf der Serverseite auch als Thread durchgeführt. Attribute wie einen Gamestate oder Lobbyliste, welche noch nicht relevant sind, wurden noch nicht implementiert. Alle Nachrichten werden in einer Queue gespeichert, wozu es eine receiveMessage und/oder handleMessage Methode braucht. Es wurde entschieden, dass alle Nachrichten vom Server und User in der Main Klasse gehandlet werden, weil dies zu weniger Komplexität führt. Der Server und die jeweiligen Threads, welche mit den Clients verbunden sind, haben unterschiedliche Aufgaben. Alles was nicht unbedingt auf den Server muss, z.B. Ping-Pong wird in den Threads gehandlet. Der Server hat nun Methoden um die Kommunikation zwischen verschiedenen Server Threads und Clients zu ermöglichen. Die neue Struktur sieht vereinfacht wie folgt aus. Klassen Message, MessageListener, ConnectionListener und DatabaseHelper werden nicht aufgeführt.

Main Klasse:
Attribute:

- Queues für Nachrichten von Server und User
- Client Objekt
- Server ID und Client Username
- Flags für Ping-Pong und online Status
- Zeit des letzten Pings

receiveMessage()
handleServerMessage()
handleClientMessage(MessageListener)
sendMessage()
startClient(ip, port)
initClient()
handlePingPong()
signInLogIn()

```
main() {
        initClient()
        signInLogIn()

        msgListener = new MessageListener(System.in)
        Thread.start(msgListener)
```

```
        while(True) {
                handlePingPong()
                handleClientMessage()
                receiveMessage()
                sendMessage()
                handleServerMessage()
        }
}
```

Client Klasse:
Attribute:

- Server Port, Addresse und ID
- Queues für Nachrichten vom User und Server
- disconnect Flag

```
sendMessages(OutputStream)
receiveMessage(MessageListener)

run() {
        socket = new Socket(serverIp, serverPort)
        iStream = socket.InputStream
        oStream = socket.OutputStream

        msgListener = new MessageListener(iStream)
        Thread.start(msgListener)

        while (!disconnect) {
                receiveMessage(msgListener)
                sendMessage(oStream)
        }

        socket.close()
        iStream.close()
        oStream.close()
}
```

Server Klasse:
Attribute:

- Queues für Nachrichten vom und zum Client
- Liste an Verbindungen
- Wert der nächsten temporären Server ID

```
startServerThread(ConnectionListener)
sendMessage()
```

```
fillReceiveMessageQueue()
handleReceiveMessage()
handleSendMessage()

main() {
        conListener = new ConnectionListener(port)
        Thread.start(conListener)

        while (true) {
                startServerThread()
                fillreceiveMessageQueue()
                handleReceiveMessage()
                handleSendMessage()
        }
}
```

Klasse ServerThread:
Attribute:

- ID und socket
- Queues für Nachrichten von Client und Server
- Flags für disconnect und Ping-Pong
- Zeit des letzten Pings
- Client Username

```
sendMessage(OutputStream)
receiveMessage(MessageListener)
handlePingPong()

run() {
        iStream = socket.InputStream
        oStream = socket.OutputStream

        msgListener = new MessageListener(iStream)
        Thread.start(msgListener)

        while (clientUsername == null) {
                receiveMessage()
                sendMessage()
        }

        while (!disconnect) {
                handlePingPong()
                receiveMessage()
                sendMessage()
```

```
        }

        socket.close()
        iStream.close()
        oStream.close()
}
```

6. Treffen 19.03.2025 12:00- 13:15, Spiegelgasse 5

Die Änderung des Codes wurden erklärt

Letzte Meilenstein 2 Aufgaben verteilt: gitignore, builder -> Alex
Readme -> Simon
Username change -> Fragen ob es automatisch überschrieben werden muss, sonst Frederic

Eventuell noch eine Log-out-Funktion hinzufügen.

Aufgaben für den 3. Meilenstein verteilen:

- Noah und Simon:
    - Proc. /** */
    - Pres. Demo! (wäre gut, rudimentär mit Client-Server)
    - Prod. Game Logic
    - Prod. Librarian (Simon)
    - Pres. QA
    - Proc. QA (advanced)
    - Pres. Rules to Codes
    - Prod. Shall We Play a Game
    - Pres. Technology! (Simon)

- Alex und Frederic:
    - Pres. About a Game
    - Prod. Broadcast
    - Prod. Build Script
    - Prod. Command Line
    - Proc. Dear Diary
    - Prod. GUI
    - Prod. Game List
    - Prod. Lounging
    - Prod. Lounging List
    - Prod. Manual
    - Prod. Player List
    - Pres. Progress Report

    - Prod. Protocol Code
    - Prod. Protocol Document
    - Prod. Whisper
    - Proc. Who? What? When? (Frederic)
    - (Bilder zum Server schicken)

- Noch nicht zugeteilt:
    - Prod. Game State

7. Treffen 26.03.2025 12:00- 13:15, Spiegelgasse 5

Schon gemacht:

Game Prototyp:
- Chat Fenster
- Zeichenfunktion
- Bewertungsphase

Server-Client:
- Interaktion durch GUI
- Erstellen von Lobbies

Idee vom Game GUI:
1. Create, Join und Overview Buttons mit Chat
2. Pop-up um Lobby Namen einzugeben
3. Alle Spieler, wenn alle bereit sind kann party leader das Spiel starten
4. Zeichenfenster mit Lobby Chat
5. Wenn die Zeit vorbei ist, wird die Zeichenfläche gesperrt und das Bild an den Server gesendet. Danach wird die Bewertungsphase aufgerufen. Bild mit Bewertungsfunktion und Lobby Chat
6. Temporäres Leaderboard
7. Am Schluss finales Leaderboard mit Play again und Exit Buttons

Netzwerkprotokoll Befehle:
- createLobby
- joinLobby
- image
- vote
- leaveLobby
- updateGame
- game

Ideen:

- Passwort als Hash speichern
- Bug/User report system
- Ban User
- Stats in der DB speichern
- Rick roll
- Cheat code
- Daily meme roll

Bonus Punkte ideen:
- 3D Grafik -> Podestdrehen
- Animation -> Charakter bewegen sich
- I/O -> Drawing pad

Keine getter setter Methoden mit JavaDoc dokumentieren

8. Treffen 03.04.2025 12:00- 13:15, Spiegelgasse 5

Issues:
- Lobby Namen Dublikate
- Random Disconnects

TODO:
- Nach Broadcast fragen (Alex)
- Build Script (Alex)
- Demo fragen (Simon)
- Game Logic (Simon & Noah)
- Logger (Alex & Frederic)
- Protocol Document (Frederic)
- QA (Alex)
- Send images (Alex & Frederic)